

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №5
по курсу «Вычислительная математика»
Вариант 19

Выполнил:

Якименко Владислав Игоревич, Р3213

Проверила:

Малышева Татьяна Алексеевна

Содержание

1	Вычислительная реализация задачи	1
1.1	Постановка задачи	1
1.2	Графическое представление	1
1.3	Рабочие формулы метода	2
1.4	Таблица конечных разностей	3
1.5	Вычисление $f(X_1)$ по второй формуле Ньютона	3
1.6	Вычисление $f(X_2)$ по первой формуле Гаусса	5
1.7	Проверка многочленом Лагранжа	6
2	Программная реализация	7
2.1	Методы, ввод и порядок работы	7
2.2	Дополнительные центральные схемы	9
2.3	Листинг программы	10
2.4	Результаты выполнения программы	15
2.5	Веб-интерфейс	17
2.6	Проверка программы	19
3	Вывод	20

1 Вычислительная реализация задачи

1.1 Постановка задачи

Цель работы: решить задачу интерполяции и найти значения функции при аргументах, отличных от узловых точек; сравнить результаты различных представлений интерполяционного многочлена.

Порядок выполнения работы:

1. Выбрать таблицу 1.4 для варианта 19 и построить таблицу конечных разностей.
2. Вычислить $f(X_1)$ подходящей формулой Ньютона, а $f(X_2)$ — формулой Гаусса.
3. Для обоих аргументов выполнить проверку многочленом Лагранжа.
4. Реализовать ввод данных, расчёт разностей, сравнение методов и графики; проверить корректные и ошибочные наборы.

$$X_1 = 1.573, \quad X_2 = 1.375, \quad h = 0.1.$$

Таблица 1: Исходные данные варианта 19 (таблица 1.4 задания)

x_i	1.05	1.15	1.25	1.35	1.45	1.55	1.65
y_i	0.1213	1.1316	2.1459	3.1565	4.1571	5.1819	6.1969

1.2 Графическое представление

Аналитическая исходная функция в задании не дана. Поэтому исходные табличные значения соединены оранжевой пунктирной линией, а полином Ньютона показан синей линией. Узлы отмечены отдельными точками.

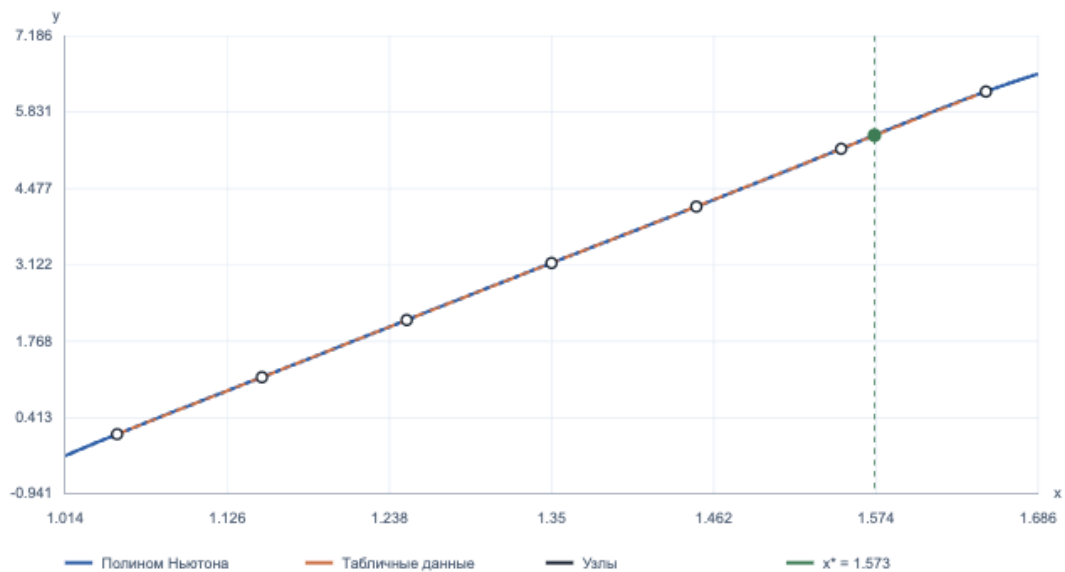


Рис. 1: Табличные данные, полином Ньютона, узлы и точка $X_1 = 1.573$

1.3 Рабочие формулы метода

Пусть заданы $n + 1$ различных узлов (x_i, y_i) , $i = 0, \dots, n$. Ищем полином P_n степени не выше n , удовлетворяющий условию $P_n(x_i) = y_i$. В данной работе $n = 6$.

Многочлен Лагранжа:

$$L_n(x) = \sum_{i=0}^n y_i \ell_i(x), \quad \ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

Разделённые разности:

$$f[x_i] = y_i, \quad f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Ньютон с разделёнными разностями (I и II):

$$N_n^I(x) = y_0 + \sum_{k=1}^n f[x_0, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j),$$
$$N_n^{II}(x) = y_n + \sum_{k=1}^n f[x_{n-k}, \dots, x_n] \prod_{j=0}^{k-1} (x - x_{n-j}).$$

Эти формулы применимы и на неравномерной сетке. Первая удобна в левой части таблицы, вторая — в правой.

Конечные разности на равномерной сетке:

$$\Delta^0 y_i = y_i, \quad \Delta^k y_i = \Delta^{k-1} y_{i+1} - \Delta^{k-1} y_i.$$

Первая формула Ньютона (вперёд):

$$N_n^I(x) = y_0 + \sum_{k=1}^n \frac{t(t-1) \cdots (t-k+1)}{k!} \Delta^k y_0, \quad t = \frac{x - x_0}{h}.$$

Вторая формула Ньютона (назад):

$$N_n^{II}(x) = y_n + \sum_{k=1}^n \frac{t(t+1) \cdots (t+k-1)}{k!} \Delta^k y_{n-k}, \quad t = \frac{x - x_n}{h}.$$

При достаточной гладкости исходной функции остаток интерполяции имеет вид

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i).$$

Для табличного варианта производная $f^{(7)}$ неизвестна, поэтому численную оценку истинной погрешности по этой формуле получить нельзя.

1.4 Таблица конечных разностей

Например,

$$\Delta y_0 = 1.1316 - 0.1213 = 1.0103,$$

$$\Delta^2 y_0 = 1.0143 - 1.0103 = 0.0040, \quad \Delta^3 y_0 = -0.0037 - 0.0040 = -0.0077.$$

Таблица 2: Таблица конечных разностей до шестого порядка

x_i	y_i	$\Delta^1 y_i$	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
1.05	0.1213	1.0103	0.0040	-0.0077	0.0014	0.0391	-0.1478
1.15	1.1316	1.0143	-0.0037	-0.0063	0.0405	-0.1087	
1.25	2.1459	1.0106	-0.0100	0.0342	-0.0682		
1.35	3.1565	1.0006	0.0242	-0.0340			
1.45	4.1571	1.0248	-0.0098				
1.55	5.1819	1.0150					
1.65	6.1969						

Разности рассчитаны до шестого порядка без промежуточного округления. Разность шестого порядка ненулевая, поэтому для использования всех исходных данных сохраняем полином шестой степени.

1.5 Вычисление $f(X_1)$ по второй формуле Ньютона

Аргумент $X_1 = 1.573$ находится в правой половине таблицы и близок к её правому концу $x_6 = 1.65$. Используем интерполирование назад:

$$t = \frac{1.573 - 1.65}{0.1} = -0.77.$$

$$\begin{aligned}
 N_6^{\Pi}(x) = & y_6 + t\Delta y_5 + \frac{t(t+1)}{2!}\Delta^2 y_4 + \frac{t(t+1)(t+2)}{3!}\Delta^3 y_3 \\
 & + \frac{t(t+1)(t+2)(t+3)}{4!}\Delta^4 y_2 \\
 & + \frac{t(t+1)(t+2)(t+3)(t+4)}{5!}\Delta^5 y_1 \\
 & + \frac{t(t+1)(t+2)(t+3)(t+4)(t+5)}{6!}\Delta^6 y_0.
 \end{aligned}$$

Из таблицы берём разности 1.0150, -0.0098 , -0.0340 , -0.0682 , -0.1087 , -0.1478 . Первые два ненулевых слагаемых:

$$(-0.77) \cdot 1.0150 = -0.78155, \quad \frac{(-0.77) \cdot 0.23}{2} \cdot (-0.0098) = 0.00086779.$$

Таблица 3: Подстановка $t = -0.77$ во вторую формулу Ньютона

k	Коэффициент	Разность	Слагаемое
0	1	6.1969	6.196900000000
1	-0.7700000000	1.0150	-0.781550000000
2	-0.0885500000	-0.0098	0.000867790000
3	-0.0363055000	-0.0340	0.001234387000
4	-0.0202403163	-0.0682	0.001380389568
5	-0.0130752443	-0.1087	0.001421279055
6	-0.0092180472	-0.1478	0.001362427381

Суммируя все семь слагаемых, получаем

$$f(1.573) \approx N_6^{\Pi}(1.573) = 5.421616273004.$$

1.6 Вычисление $f(X_2)$ по первой формуле Гаусса

Центральный узел $a = x_3 = 1.35$. Здесь удобно переобозначить узлы: $z_j = x_{j+3}$, $v_j = y_{j+3}$, $j = -3, \dots, 3$. Тогда $z_0 = a$, $v_0 = 3.1565$, а

$$X_2 = 1.375 > a, \quad t = \frac{1.375 - 1.35}{0.1} = 0.25.$$

Следовательно, используем **первую формулу Гаусса**:

$$\begin{aligned} G_6^I(x) = & v_0 + t\Delta v_0 + \frac{t(t-1)}{2!}\Delta^2 v_{-1} + \frac{t(t-1)(t+1)}{3!}\Delta^3 v_{-1} \\ & + \frac{t(t-1)(t+1)(t-2)}{4!}\Delta^4 v_{-2} \\ & + \frac{t(t-1)(t+1)(t-2)(t+2)}{5!}\Delta^5 v_{-2} \\ & + \frac{t(t-1)(t+1)(t-2)(t+2)(t-3)}{6!}\Delta^6 v_{-3}. \end{aligned}$$

Порядок добавления узлов: $z_0, z_1, z_{-1}, z_2, z_{-2}, z_3, z_{-3}$. Из исходной таблицы нужны

$$\Delta y_3 = 1.0006, \quad \Delta^2 y_2 = -0.0100, \quad \Delta^3 y_2 = 0.0342,$$

$$\Delta^4 y_1 = 0.0405, \quad \Delta^5 y_1 = -0.1087, \quad \Delta^6 y_0 = -0.1478.$$

Например,

$$\begin{aligned} \frac{0.25(-0.75)}{2}(-0.01) &= 0.0009375, \\ \frac{0.25(-0.75)(1.25)}{6}(0.0342) &= -0.0013359375. \end{aligned}$$

Таблица 4: Подстановка $t = 0.25$ в первую формулу Гаусса

k	Коэффициент	Разность	Слагаемое
0	1	3.1565	3.156500000000
1	0.2500000000	1.0006	0.250150000000
2	-0.0937500000	-0.0100	0.000937500000
3	-0.0390625000	0.0342	-0.001335937500
4	0.0170898438	0.0405	0.000692138672
5	0.0076904297	-0.1087	-0.000835949707
6	-0.0035247803	-0.1478	0.000520962524

Сумма слагаемых:

$$f(1.375) \approx G_6^I(1.375) = 3.406628713989.$$

1.7 Проверка многочленом Лагранжа

Для равномерной сетки вводим $s = (x - 1.05)/0.1$, тогда

$$\ell_i(x) = \frac{\prod_{j \neq i} (s - j)}{\prod_{j \neq i} (i - j)}, \quad \prod_{j \neq i} (i - j) = (-1)^{6-i} i! (6 - i)!.$$

Знаменатели для $i = 0, \dots, 6$: 720, -120, 48, -36, 48, -120, 720. При X_1 получаем $s_1 = 5.23$, при X_2 — $s_2 = 3.25$. Например, для центрального базиса в точке X_2 :

$$\ell_3(1.375) = \frac{3.25 \cdot 2.25 \cdot 1.25 \cdot (-0.75) \cdot (-1.75) \cdot (-2.75)}{-36} = 0.916442871094.$$

Таблица 5: Слагаемые многочлена Лагранжа при $x = 1.573$

i	$\ell_i(x)$	$y_i \ell_i(x)$
0	-0.009218047230	-0.001118149129
1	0.068383527676	0.077382799918
2	-0.223887246184	-0.480439641585
3	0.432380152570	1.364807951586
4	-0.587931548921	-2.444090242020
5	1.257662269866	6.517080116218
6	0.062610892223	0.387993438015

Таблица 6: Слагаемые многочлена Лагранжа при $x = 1.375$

i	$\ell_i(x)$	$y_i \ell_i(x)$
0	-0.003524780273	-0.000427555847
1	0.030548095703	0.034568225098
2	-0.137466430664	-0.294989213562
3	0.916442871094	2.892751922607
4	0.229110717773	0.952436164856
5	-0.039276123047	-0.203524942017
6	0.004165649414	0.025814112854

В каждой таблице $\sum_i \ell_i(x) = 1$; сумма последнего столбца даёт

$$L_6(1.573) = 5.421616273004, \quad L_6(1.375) = 3.406628713989.$$

Результаты совпали с выбранными формулами Ньютона и Гаусса. В таблицах отображены округлённые числа; суммирование выполнялось до округления.

2 Программная реализация

Проект написан на JavaScript и запускается в Node.js 20 или новее. Консоль и браузер используют общий модуль `src/interpolation.js`. Внешние библиотеки для интерполяции не используются.

2.1 Методы, ввод и порядок работы

Согласно таблице 2 задания для варианта 19 реализованы методы 1, 2, 3: Лагранж, Ньютон с разделёнными разностями (I и II) и Ньютон с конечными разностями (I и II). Для проверки вычислительной части добавлены обе формулы Гаусса. Выполнено также необязательное задание: схемы Стирлинга и Бесселя.

Предусмотрены три способа задания исходных данных:

1. Ввод пар (x, y) с клавиатуры в консоли или редактирование таблицы в браузере.
2. Загрузка JSON (`points` либо массивы x, y) или текстового файла с двумя числами в строке. Разделители — пробел и точка с запятой; допускается десятичная запятая.
3. Табулирование одной из функций $\sin x$, e^x , $x^3 - 2x + 1$: пользователь выбирает интервал, число точек и аргумент интерполяции.

Узлы сортируются по x с сохранением пар (x, y) . Проверяются число точек (от 2 до 50), конечность значений, отсутствие совпадающих или машинно неразличимых абсцисс и равномерность сетки. Ограничение 50 узлов принято в реализации, а не перенесено из задания. Неравномерная сетка допустима для Лагранжа и Ньютона с разделёнными разностями; остальные методы получают диагностику неприменимости.

Программа выводит обе таблицы разностей на равномерной сетке, результат каждого метода, рекомендуемое направление Ньютона и максимальное расхождение применимых методов. Если выбрана известная функция, дополнительно вычисляется $|P(x) - f(x)|$. Выход за интервал помечается как экстраполяция; для большой степени полинома выводится предупреждение о численной неустойчивости.

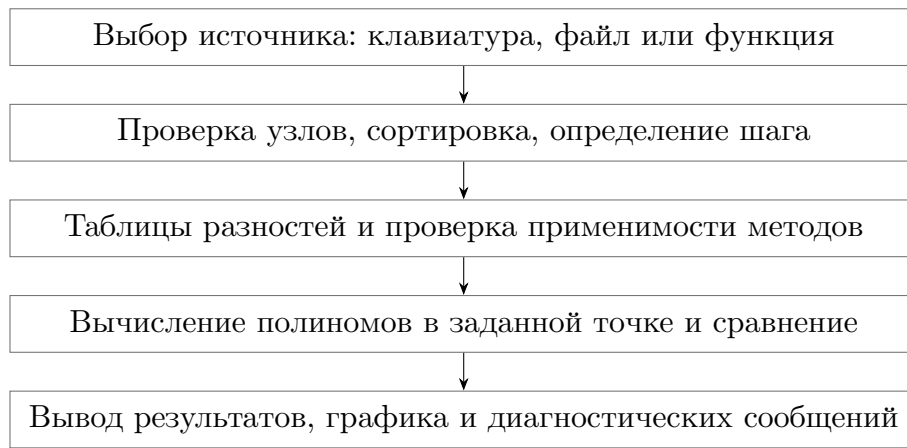


Рис. 2: Основные этапы алгоритма интерполяции

2.2 Дополнительные центральные схемы

В данной реализации центральные формулы используют полный набор узлов. Гаусс и Стирлинг реализованы для нечётного числа равноотстоящих узлов, Бессель — для чётного. Программа не отбрасывает крайние узлы незаметно для пользователя, поэтому на семи узлах варианта 19 Бессель помечается как неприменимый. Его проверка выполняется отдельно на шести узлах.

Для второй формулы Гаусса порядок узлов: $z_0, z_{-1}, z_1, z_{-2}, z_2, \dots$. Первые слаемые имеют вид

$$G^{\text{II}}(x) = v_0 + t\Delta v_{-1} + \frac{t(t+1)}{2!}\Delta^2 v_{-1} + \frac{t(t+1)(t-1)}{3!}\Delta^3 v_{-2} + \dots$$

Её обычно выбирают при $x < a$. В точной арифметике обе полные схемы Гаусса задают тот же полином, что и Лагранж.

Стирлинг. Для $2m+1$ узлов $z_{-m}, \dots, z_m$, $t = (x - z_0)/h$ и $A_r(t) = \prod_{j=1}^{r-1}(t^2 - j^2)$:

$$S_{2m}(x) = v_0 + \sum_{r=1}^m \left[\frac{tA_r(t)}{(2r-1)!} \frac{\Delta^{2r-1}v_{-r} + \Delta^{2r-1}v_{-r+1}}{2} + \frac{t^2A_r(t)}{(2r)!} \Delta^{2r}v_{-r} \right].$$

Формула является средним двух формул Гаусса; рекомендуется при $|t| \leq 0.25$. В чётных слагаемых стоит t^2 , что также следует из симметрии. Для X_2 получаем $t = 0.25$ и $S_6(X_2) = 3.406628713989$.

Бессель. Для $2m+2$ узлов $z_{-m}, \dots, z_{m+1}$, $t = (x - z_0)/h$ и $B_r(t) = \prod_{j=0}^{r-1}(t+j)(t-j-1)$:

$$B_{2m+1}(x) = \frac{v_0 + v_1}{2} + (t - \frac{1}{2})\Delta v_0 + \sum_{r=1}^m \left[\frac{B_r(t)}{(2r)!} \frac{\Delta^{2r}v_{-r} + \Delta^{2r}v_{-r+1}}{2} + \frac{(t - \frac{1}{2})B_r(t)}{(2r+1)!} \Delta^{2r+1}v_{-r} \right].$$

Рекомендуется $0.25 \leq t \leq 0.75$. Для проверки использована функция $f(x) = x^3 - 2x + 1$ на шести узлах:

$$-1, \quad -0.5, \quad 0, \quad 0.5, \quad 1, \quad 1.5.$$

Центральная пара — $0, 0.5$. При $x = 0.25$, $t = 0.5$ схема Бесселя даёт 0.515625 , что совпадает с $f(0.25)$.

2.3 Листинг программы

Ниже приведён расчётный модуль целиком: исходные данные, проверка входа, таблицы разностей, все методы, табулирование и сравнение результатов. Остальные файлы проекта содержат консольный ввод, загрузку файлов, генерацию SVG и веб-интерфейс.

```
1 import { parseNumber } from "./utils.js";
2
3 export const VARIANT19 = {
4   points: [0.1213, 1.1316, 2.1459, 3.1565, 4.1571, 5.1819, 6.1969]
5     .map((y, i) => ({ x: (105 + 10 * i) / 100, y })),
6   targets: [1.573, 1.375],
7 };
8
9 export const FUNCTIONS = [
10   { id: "sin", label: "sin(x)", evaluate: Math.sin },
11   { id: "exp", label: "exp(x)", evaluate: Math.exp },
12   { id: "cubic", label: "x3 - 2x + 1", evaluate: x => x ** 3 - 2 * x + 1 },
13 ];
14
15 export function validatePoints(input) {
16   if (!Array.isArray(input) || input.length < 2 || input.length > 50) {
17     throw new Error("Таблица должна содержать от 2 до 50 точек.");
18   }
19   const points = input.map((p, i) => {
20     if (!p || typeof p !== "object" || (Array.isArray(p) && p.length !== 2)) {
21       throw new ErrorСтрока(` ${i + 1}: требуется пара x, y.`);
22     }
23     return { x: parseNumber(Array.isArray(p) ? p[0] : p.x, `x в строке ${i + 1}`),
24       y: parseNumber(Array.isArray(p) ? p[1] : p.y, `y в строке ${i + 1}`) };
25   }).sort((a, b) => a.x - b.x);
26   for (let i = 1; i < points.length; i++) {
27     const scale = Math.max(Math.abs(points[i].x), Math.abs(points[i - 1].x), Number.
28       MIN_VALUE);
29     if (points[i].x - points[i - 1].x <= 32 * Number.EPSILON * scale) {
30       throw new Error("Узлы x должны быть различными и различимыми в машинной точности.");
31     }
32   }
33   return points;
34 }
35
36 export function uniformStep(points) {
37   const h = (points.at(-1).x - points[0].x) / (points.length - 1);
38   if (!Number.isFinite(h) || h <= 0) return null;
39   const scale = Math.max(...points.map(p => Math.abs(p.x)), h);
40   const tolerance = Math.max(h * 1e-10, 16 * Number.EPSILON * scale);
41   return points.every((p, i) => Math.abs(p.x - (points[0].x + i * h)) <= tolerance) ? h :
42     null;
43 }
```

```

42
43 // Columns: table[k][i] is the difference of order k starting at node i.
44 export function differenceTable(points, divided = false) {
45   const table = [points.map(p => p.y)];
46   for (let k = 1; k < points.length; k++) {
47     table.push(Array.from({ length: points.length - k }, (_, i) => {
48       const difference = table[k - 1][i + 1] - table[k - 1][i];
49       const value = divided ? difference / (points[i + k].x - points[i].x) : difference;
50       if (!Number.isFinite(value)) throw new Error("Переполнение таблицы разностей.
Уменьшите масштаб данных.");
51       return value;
52     }));
53   }
54   return table;
55 }
56
57 export function lagrange(points, x) {
58   return points.reduce((sum, point, i) => {
59     let basis = 1;
60     for (let j = 0; j < points.length; j++) {
61       if (i !== j) basis *= (x - points[j].x) / (point.x - points[j].x);
62     }
63     return sum + point.y * basis;
64   }, 0);
65 }
66
67 export function newtonDivided(points, table, x, backward = false) {
68   const n = points.length - 1;
69   let result = table[n][0];
70   for (let k = n - 1; k >= 0; k--) {
71     const index = backward ? n - k : 0;
72     const node = backward ? n - k : k;
73     result = table[k][index] + (x - points[node].x) * result;
74   }
75   return result;
76 }
77
78 export function newtonFinite(points, table, x, backward = false) {
79   const h = uniformStep(points);
80   if (h === null) throw new Error("Нужна равномерная сетка.");
81   const n = points.length - 1;
82   const t = (x - points[backward ? n : 0].x) / h;
83   let factor = 1;
84   let result = table[0][backward ? n : 0];
85   for (let k = 1; k <= n; k++) {
86     factor *= (t + (backward ? k - 1 : 1 - k)) / k;
87     result += factor * table[k][backward ? n - k : 0];
88   }
89   return result;
90 }

```

```

91
92 export function gauss(points, table, x, backward = false) {
93   const h = uniformStep(points);
94   if (h === null || points.length % 2 !== 1 || points.length < 3) {
95     throw new Error("Для полной схемы Гаусса нужны равномерная сетка и нечётное число
96       узлов не менее 3.");
97   }
98   const center = (points.length - 1) / 2;
99   const t = (x - points[center].x) / h;
100   let result = table[0][center];
101   let factor = 1;
102   for (let k = 1; k < points.length; k++) {
103     const j = k - 1;
104     const offset = j === 0 ? 0 : (j % 2 === 1 ? (j + 1) / 2 : -j / 2);
105     factor *= (t - (backward ? -offset : offset)) / k;
106     const index = center - (backward ? Math.ceil(k / 2) : Math.floor(k / 2));
107     result += factor * table[k][index];
108   }
109   return result;
110 }
111
112 export function stirling(points, table, x) {
113   const h = uniformStep(points);
114   if (h === null || points.length % 2 !== 1 || points.length < 3) {
115     throw new Error("Стирлинг: нужны равномерная сетка и нечётное число узлов не менее 3.
116       ");
117   }
118   const m = (points.length - 1) / 2;
119   const t = (x - points[m].x) / h;
120   let result = table[0][m];
121   let product = 1;
122   let factorial = 1;
123   for (let r = 1; r <= m; r++) {
124     factorial *= 2 * r - 1;
125     result += t * product / factorial * (table[2 * r - 1][m - r] + table[2 * r - 1][m - r
126       + 1]) / 2;
127     factorial *= 2 * r;
128     result += t * t * product / factorial * table[2 * r][m - r];
129     product *= t * t - r * r;
130   }
131   return result;
132 }
133
134 export function bessel(points, table, x) {
135   const h = uniformStep(points);
136   if (h === null || points.length % 2 !== 0) {
137     throw new Error("Бессель: нужны равномерная сетка и чётное число узлов.");
138   }
139   const m = points.length / 2 - 1;
140   const t = (x - points[m].x) / h;

```

```

138 let result = (table[0][m] + table[0][m + 1]) / 2 + (t - 0.5) * table[1][m];
139 let product = 1;
140 let factorial = 1;
141 for (let r = 1; r <= m; r++) {
142     product *= (t + r - 1) * (t - r);
143     factorial *= 2 * r;
144     result += product / factorial * (table[2 * r][m - r] + table[2 * r][m - r + 1]) / 2;
145     factorial *= 2 * r + 1;
146     result += (t - 0.5) * product / factorial * table[2 * r + 1][m - r];
147 }
148 return result;
149 }
150
151 export function generatePoints(id, a, b, count) {
152     const fn = FUNCTIONS.find(item => item.id === id);
153     if (!fn) throw new Error("Неизвестная функция. Доступны sin, exp, cubic.");
154     a = parseNumber(a, "Начало интервала");
155     b = parseNumber(b, "Конец интервала");
156     count = parseNumber(count, "Число точек");
157     if (!(a < b) || !Number.isFinite(b - a)) throw new Error("Нужен конечный интервал a < b");
158     if (!Number.isInteger(count) || count < 2 || count > 50) throw new Error("Число точек должно быть целым от 2 до 50.");
159     return validatePoints(Array.from({ length: count }, (_, i) => {
160         const x = i === count - 1 ? b : a + (b - a) * i / (count - 1);
161         return { x, y: fn.evaluate(x) };
162     }));
163 }
164
165 export function solve(input, target, functionId = null) {
166     const points = validatePoints(input);
167     const x = parseNumber(target, "Аргумент интерполяции");
168     const h = uniformStep(points);
169     const divided = differenceTable(points, true);
170     const finite = h === null ? null : differenceTable(points);
171     const fn = functionId ? FUNCTIONS.find(item => item.id === functionId) : null;
172     if (functionId && !fn) throw new Error("Неизвестная исходная функция.");
173     const warnings = [];
174     if (x < points[0].x || x > points.at(-1).x) warnings.push("Аргумент вне интервала: выполняется экстраполяция.");
175     if (points.length > 15) warnings.push("Высокая степень полинома: возможны осцилляции и накопление ошибок округления.");
176     if (h === null) warnings.push("Сетка неравномерная: методы с конечными разностями неприменимы.");
177     const definitions = [
178         ["lagrange", "Лагранж", z => lagrange(points, z)],
179         ["divided-forward", "Ньютон: разделённые, I", z => newtonDivided(points, divided, z)
180     ],
181     ["divided-backward", "Ньютон: разделённые, II", z => newtonDivided(points, divided, z, true)],

```

```

181 ["finite-forward", "Ньютон: конечные, I", z => newtonFinite(points, finite, z)],
182 ["finite-backward", "Ньютон: конечные, II", z => newtonFinite(points, finite, z, true
)],
183 ["gauss-forward", "Гаусс, I", z => gauss(points, finite, z)],
184 ["gauss-backward", "Гаусс, II", z => gauss(points, finite, z, true)],
185 ["stirling", "Стирлинг", z => stirling(points, finite, z)],
186 ["bessel", "Бессель", z => bessel(points, finite, z)],
187 ];
188 const models = definitions.map(([id, label, evaluate], i) => {
189   try {
190     const value = evaluate(x);
191     if (!Number.isFinite(value)) throw new Error("Переполнение при вычислении полинома.
");
192     return { id, label, evaluate, value, applicable: true, extra: i >= 5 };
193   } catch (error) {
194     return { id, label, applicable: false, reason: error.message, extra: i >= 5 };
195   }
196 });
197 const valid = models.filter(m => m.applicable);
198 if (!valid.some(m => !m.extra)) throw new Error("Вычисление невозможно в машинной
точности. Измените масштаб данных.");
199 const preferred = x <= points[0].x + (points.at(-1).x - points[0].x) / 2 ? "I вперёд()"
: "II назад()";
200 const spread = Math.max(...valid.map(m => m.value)) - Math.min(...valid.map(m => m.
value));
201 if (spread > 1e-8 * Math.max(1, ...valid.map(m => Math.abs(m.value)))) warnings.push("
Заметное расхождение методов изза- численной неустойчивости. Уменьшите число узлов
или масштаб данных.");
202 const exact = fn?.evaluate(x);
203 if (fn && !Number.isFinite(exact)) warnings.push("Исходная функция не имеет конечного
машинного значения в выбранной точке.");
204 return { points, x, h, divided, finite, models, preferred, spread, warnings,
205   functionId, exact: Number.isFinite(exact) ? exact : null };
206 }

```


2.4 Результаты выполнения программы

Таблица 7: Сравнение методов на исходном наборе варианта 19

Метод	$X_1 = 1.573$	$X_2 = 1.375$
Лагранж	5.421616273004	3.406628713989
Ньютон, разделённые I	5.421616273004	3.406628713989
Ньютон, разделённые II	5.421616273004	3.406628713989
Ньютон, конечные I	5.421616273004	3.406628713989
Ньютон, конечные II	5.421616273004	3.406628713989
Гаусс I	5.421616273004	3.406628713989
Гаусс II	5.421616273004	3.406628713989
Стирлинг	5.421616273004	3.406628713989
Бессель	неприменим: нечётное число узлов	

Расхождение применимых методов при запуске в Node.js составило $3.55 \cdot 10^{-15}$ для X_1 и $2.22 \cdot 10^{-15}$ для X_2 . Это эффект округления в арифметике двойной точности, а не оценка погрешности относительно неизвестной функции.

На неравномерном наборе $x = -1, -0.2, 0.5, 1.7, 3$ со значениями $y = x^2 - 2x + 3$ три применимых метода дают $P(0.7) = 2.09$. Конечные разности для интерполяционных формул не используются.

Для $\sin x$ на $[0; \pi]$ при $x = 0.7$ программа выдаёт:

Таблица 8: Интерполяция известной функции

Число узлов	$P(0.7)$	$\sin(0.7)$	Абсолютная ошибка
5	0.643980031429	0.644217687238	0.000237655809
7	0.644212684822	0.644217687238	0.000005002415

Увеличение числа узлов с 5 до 7 в этом опыте уменьшило абсолютную погрешность. Это не универсальное правило: полиномы высокой степени могут осциллировать, особенно у концов равномерной сетки.

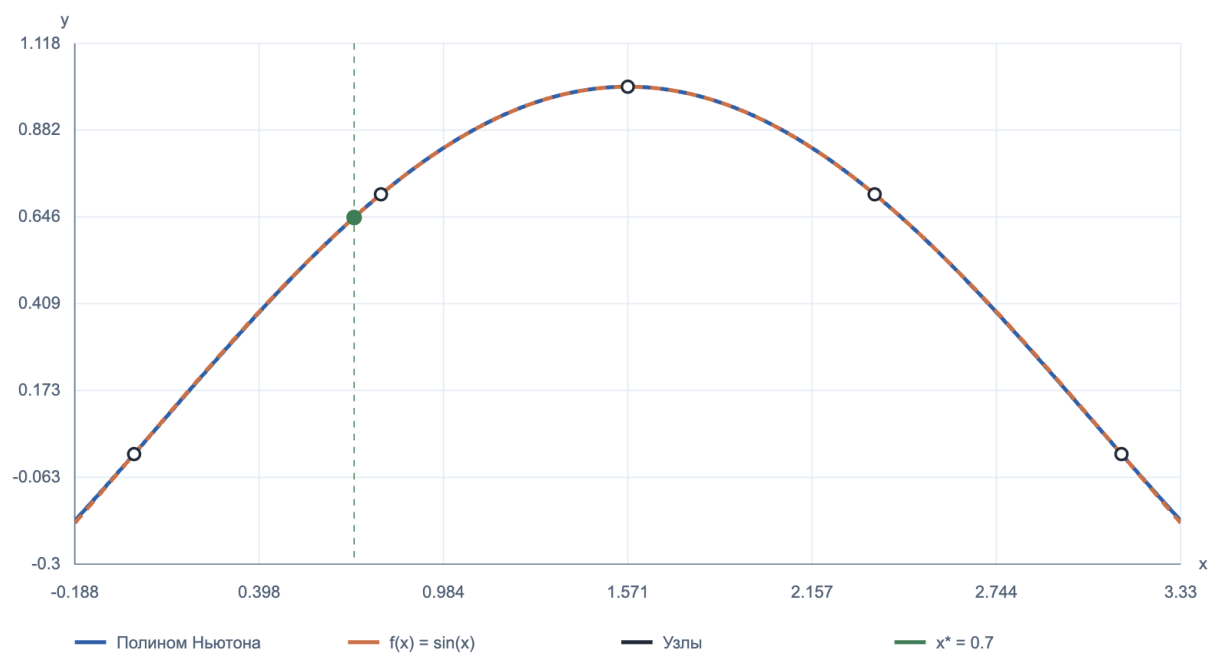


Рис. 3: Интерполяция $\sin x$ по пяти узлам: функция и полином показаны разными цветами

2.5 Веб-интерфейс

Веб-интерфейс содержит левую панель ввода и правую область результатов. Доступны редактирование узлов, загрузка файлов, табулирование функции, выбор аргумента, просмотр разностей и скачивание текстового результата или SVG. График поддерживает масштабирование колесом и кнопками, перемещение перетаскиванием и сброс масштаба.

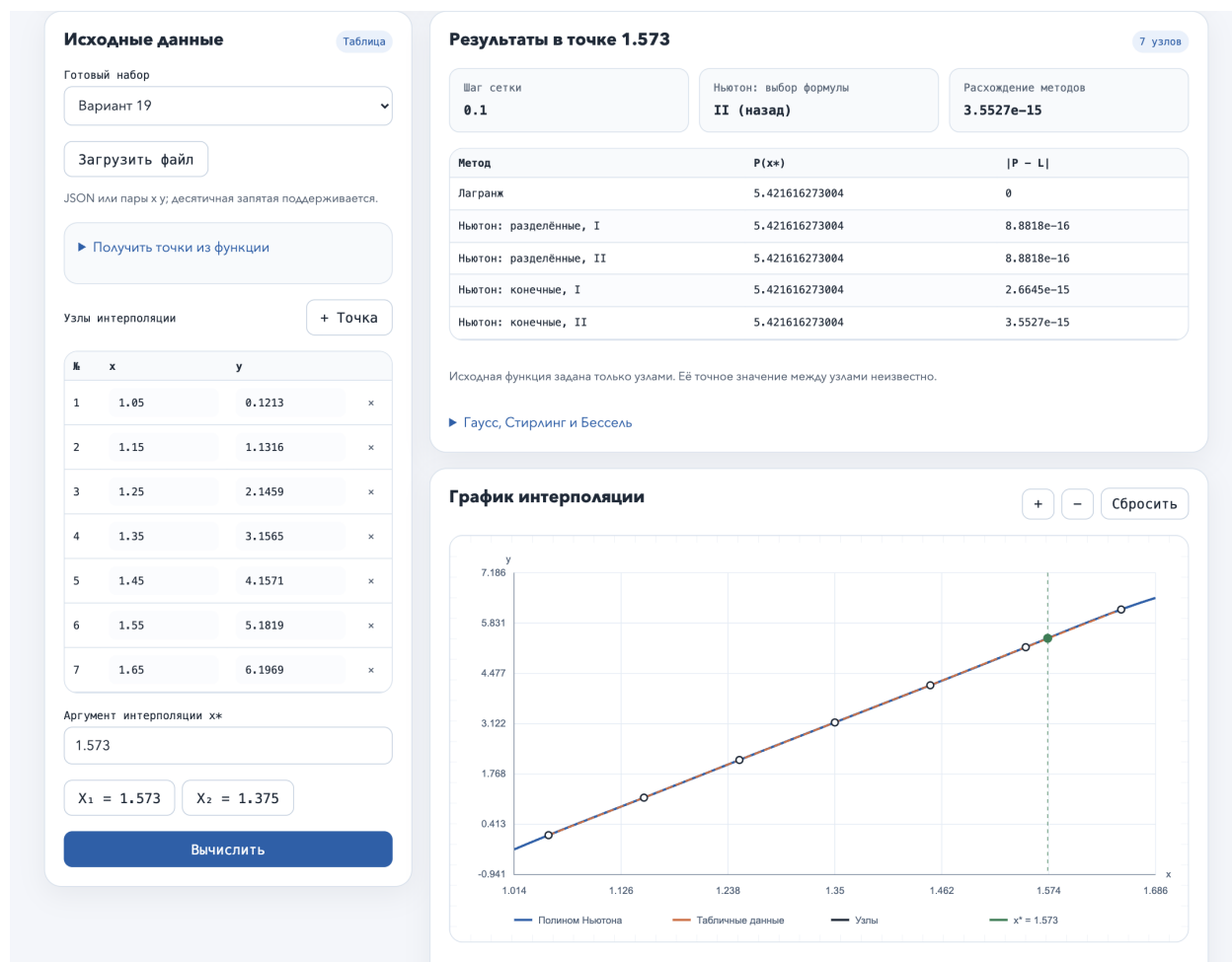


Рис. 4: Веб-интерфейс ЛР №5 в широком окне

Исходные данные

Таблица

Готовый набор

Вариант 19

Загрузить файл

JSON или пары x y ; десятичная запятая поддерживается.

► Получить точки из функции

Узлы интерполяции

+ Точка

№	x	y	
1	1.05	0.1213	×
2	1.15	1.1316	×
3	1.25	2.1459	×
4	1.35	3.1565	×
5	1.45	4.1571	×
6	1.55	5.1819	×
7	1.65	6.1969	×

Аргумент интерполяции x^*

1.573

$x_1 = 1.573$

$x_2 = 1.375$

Вычислить

Рис. 5: Адаптивная форма в окне шириной 390 пикселей

2.6 Проверка программы

Подготовлены четыре корректных JSON-набора, текстовый набор с десятичной запятой и два некорректных файла.

Таблица 9: Контрольные наборы и фактические результаты

Набор	Проверка	Результат
Вариант 19	7 узлов, два аргумента	Совпадение с вычислительной частью
Неравномерная сетка	$y = x^2 - 2x + 3$, $x = 0.7$	2.09; конечные схемы неприменимы
Кубический полином	6 узлов, $x = 0.25$	0.515625, включая Бесселя
Синус	7 узлов на $[0; \pi]$, $x = 0.7$	Сравнение с $\sin(0.7)$, вывод ошибки
Десятичная запятая	Текст с разделителем «;»	Успешное чтение чисел
Повторяющийся x	Две точки с $x = 0$	Диагностика различных узлов
Одна точка	Недостаточный размер таблицы	Ошибка: требуется от 2 до 50 точек

Все 21 автоматический тест завершились успешно. Проверяются значения варианта, воспроизведение полиномов степеней 1–8, прохождение через каждый узел, неравномерная сетка, сортировка, малые масштабы, форматы файлов, нечисловые и бесконечные значения, повторяющиеся узлы, некорректные интервалы, табулирование трёх функций, экстраполяция и работа консоли, включая ввод с клавиатуры. Контрольные значения варианта дополнительно получены независимым расчётом в рациональной арифметике.

В браузере проверены исходный расчёт, переход к X_2 , изменение масштаба и сброс, неравномерный набор, чётный набор для Бесселя, генерация функции, диагностика повторяющихся узлов, загрузка файла и скачивание результата. В окне 390 пикселей таблицы прокручиваются внутри своих блоков; горизонтального переполнения страницы нет. Ошибок JavaScript не обнаружено.

3 Вывод

Для таблицы варианта 19 построены конечные разности до шестого порядка. По второй формуле Ньютона получено $f(1.573) \approx 5.421616273004$, по первой формуле Гаусса — $f(1.375) \approx 3.406628713989$. Многочлен Лагранжа дал те же значения; максимальное расхождение результатов составило $3.55 \cdot 10^{-15}$. Программа выводит таблицы конечных и разделённых разностей, вычисляет значения интерполяционного многочлена и строит его график. Все 21 автоматический тест завершились без ошибок.